PYTHON PARA VIDEOJUEGOS CON PYGAME · LECCIÓN 7

El jefe final y tu propio mod

Objetivo: Al terminar puedes mover un personaje en círculo con seno y coseno, agregar un jefe con vida propia y una condición de victoria, y leer el código del juego original para modificarlo.
Antes de empezar:

- Lección 6 - el juego con menú, pausa y game over.

Para copiar el código: al copiar desde un PDF, muchos visores pierden los espacios del principio de cada línea, y en Python esos espacios importan. En la misma carpeta de esta guía está cada paso como archivo `.py`: ábrelo y copia de ahí.

Paso a paso

El jefe final y tu propio mod

Última lección: tu juego necesita un final, y todo buen final tiene un jefe. Primero vas a hacer que el jefe se mueva en círculos, con un poco de matemática. Después le das vida, y una forma de ganarle. Y al final abres el juego original, Most Wanted UFO Classic, para reconocer lo que aprendiste y modificarlo.

El jefe es otro Sprite, con su propio ángulo y su propia vida

Empecemos por arriba. En la primera línea, importa math, el módulo de matemáticas de Python: de ahí vas a sacar el seno y el coseno. Junto a las demás imágenes, carga la del jefe, boss_image, con convert_alpha, para que su fondo transparente siga siendo transparente. Después de la clase Enemy, crea la clase Boss, que también es un Sprite. scale2x agranda su imagen al doble, para que el jefe imponga, y get_rect le da un rectángulo para ubicarlo y para detectar choques. Además guarda dos datos propios: un ángulo que empieza en cero, y diez puntos de vida, porque a un jefe no se le gana de un solo disparo.

Tu `game.py` queda así — las líneas resaltadas son las nuevas o cambiadas (archivo `paso-1.py`):

```
import math
import random

import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
```

```
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
boss_image = pygame.image.load("assets/boss.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
hit_sound = pygame.mixer.Sound("assets/collision.wav")
lose_sound = pygame.mixer.Sound("assets/loselife.wav")
font = pygame.font.Font(None, 36)
big_font = pygame.font.Font(None, 80)


class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)

    def update(self):
        self.rect.y += self.speed


class Boss(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = pygame.transform.scale2x(boss_image)
        self.rect = self.image.get_rect()
        self.angle = 0
        self.health = 10


def draw_centered(text, font, y):
    image = font.render(text, True, (255, 255, 255))
    screen.blit(image, image.get_rect(center=(400, y)))


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))
score = 0
lives = 3
state = "menu"

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RETURN and state not in ("playing", "paused"):
                state = "playing"
                player.midbottom = (400, 590)
                score = 0
                lives = 3
                bullets = []
                for enemy in enemies:
```

```
                    enemy.reset()
            elif event.key == pygame.K_p and state == "playing":
                state = "paused"
            elif event.key == pygame.K_p and state == "paused":
                state = "playing"

    if state == "playing":
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT]:
            player.x -= speed
        if keys[pygame.K_RIGHT]:
            player.x += speed
        player.clamp_ip(screen.get_rect())

        if cooldown > 0:
            cooldown -= 1
        if keys[pygame.K_SPACE] and cooldown == 0:
            bullets.append(bullet_image.get_rect(midbottom=player.midtop))
            shoot_sound.play()
            cooldown = 15

        for bullet in bullets:
            bullet.y -= 10
        bullets = [b for b in bullets if b.bottom > 0]
        enemies.update()

        for enemy in enemies:
            hit = enemy.rect.collidelist(bullets)
            if hit != -1:
                bullets.pop(hit)
                enemy.reset()
                score += 10
                hit_sound.play()
            elif enemy.rect.top > 600 or enemy.rect.colliderect(player):
                enemy.reset()
                lives -= 1
                lose_sound.play()
        if lives <= 0:
            state = "game_over"

    screen.blit(background, (0, 0))
    if state == "menu":
        draw_centered("Most Wanted UFO Mini", big_font, 250)
        draw_centered("Press ENTER to play", font, 340)
    else:
        enemies.draw(screen)
        for bullet in bullets:
            screen.blit(bullet_image, bullet)
        screen.blit(player_image, player)
        hud = font.render(f"Score: {score}   Lives: {lives}", True, (255, 255, 255))
        screen.blit(hud, (10, 10))
    if state == "paused":
        draw_centered("PAUSED", big_font, 300)
    if state == "game_over":
        draw_centered("GAME OVER", big_font, 250)
        draw_centered(f"Score: {score}  -  ENTER to play again", font, 340)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Coseno y seno de un ángulo que crece dibujan un círculo

Ahora, dentro de la clase Boss, agrega el método update. Ahí está el truco: en cada cuadro, el ángulo crece tres centésimas. Ese ángulo dice cuánto avanzó el jefe en su vuelta, y una vuelta completa mide unos seis coma dos ocho. Imagina un punto que da vueltas en un círculo. El coseno del ángulo dice qué tan a la derecha o a la izquierda está, de menos uno a uno, y el seno dice qué tan arriba o abajo. Multiplicas el coseno por doscientos cincuenta y el seno por sesenta, y los sumas al punto cuatrocientos, ciento cincuenta: el jefe recorre un óvalo ancho y bajito, en la parte de arriba de la pantalla. A sesenta cuadros por segundo, completa una vuelta cada tres segundos y medio.

Tu `game.py` queda así — las líneas resaltadas son las nuevas o cambiadas (archivo `paso-2.py`):

```
import math
import random

import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
boss_image = pygame.image.load("assets/boss.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
hit_sound = pygame.mixer.Sound("assets/collision.wav")
lose_sound = pygame.mixer.Sound("assets/loselife.wav")
font = pygame.font.Font(None, 36)
big_font = pygame.font.Font(None, 80)


class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)

    def update(self):
        self.rect.y += self.speed


class Boss(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = pygame.transform.scale2x(boss_image)
        self.rect = self.image.get_rect()
        self.angle = 0
        self.health = 10
```

```
    def update(self):
        self.angle += 0.03
        self.rect.center = (400 + math.cos(self.angle) * 250,
                            150 + math.sin(self.angle) * 60)


def draw_centered(text, font, y):
    image = font.render(text, True, (255, 255, 255))
    screen.blit(image, image.get_rect(center=(400, y)))


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))
score = 0
lives = 3
state = "menu"

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RETURN and state not in ("playing", "paused"):
                state = "playing"
                player.midbottom = (400, 590)
                score = 0
                lives = 3
                bullets = []
                for enemy in enemies:
                    enemy.reset()
            elif event.key == pygame.K_p and state == "playing":
                state = "paused"
            elif event.key == pygame.K_p and state == "paused":
                state = "playing"

    if state == "playing":
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT]:
            player.x -= speed
        if keys[pygame.K_RIGHT]:
            player.x += speed
        player.clamp_ip(screen.get_rect())

        if cooldown > 0:
            cooldown -= 1
        if keys[pygame.K_SPACE] and cooldown == 0:
            bullets.append(bullet_image.get_rect(midbottom=player.midtop))
            shoot_sound.play()
            cooldown = 15

        for bullet in bullets:
            bullet.y -= 10
        bullets = [b for b in bullets if b.bottom > 0]
        enemies.update()
```

```
        for enemy in enemies:
            hit = enemy.rect.collidelist(bullets)
            if hit != -1:
                bullets.pop(hit)
                enemy.reset()
                score += 10
                hit_sound.play()
            elif enemy.rect.top > 600 or enemy.rect.colliderect(player):
                enemy.reset()
                lives -= 1
                lose_sound.play()
        if lives <= 0:
            state = "game_over"

    screen.blit(background, (0, 0))
    if state == "menu":
        draw_centered("Most Wanted UFO Mini", big_font, 250)
        draw_centered("Press ENTER to play", font, 340)
    else:
        enemies.draw(screen)
        for bullet in bullets:
            screen.blit(bullet_image, bullet)
        screen.blit(player_image, player)
        hud = font.render(f"Score: {score}   Lives: {lives}", True, (255, 255, 255))
        screen.blit(hud, (10, 10))
    if state == "paused":
        draw_centered("PAUSED", big_font, 300)
    if state == "game_over":
        draw_centered("GAME OVER", big_font, 250)
        draw_centered(f"Score: {score}  -  ENTER to play again", font, 340)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

El jefe aparece a los cien puntos, recibe daño y al llegar a cero ganas

Ahora, la pelea. Antes del bucle, junto a las vidas, crea boss con el valor None, que en Python significa nada: al empezar, todavía no hay jefe. En el bloque de Enter que empieza una partida, vuelve a poner boss en None. Si no lo haces, la partida nueva empezaría con el jefe de la anterior. Ahora la lógica, dentro de playing, justo después de los choques con los enemigos. Si todavía no hay jefe y llegas a cien puntos, lo creas; revisar que sea None evita crear uno nuevo en cada cuadro. Si el jefe existe, lo mueves con update, y con collidelist buscas si alguna bala lo toca. Si una lo toca, esa bala desaparece, el jefe pierde un punto de vida, y suena el golpe. Cuando su vida llega a cero, ganas quinientos puntos, y state pasa a won, así que la partida se detiene.

Tu `game.py` queda así — las líneas resaltadas son las nuevas o cambiadas (archivo `paso-3.py`):

```
import math
import random

import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()
```

```
background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
boss_image = pygame.image.load("assets/boss.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
hit_sound = pygame.mixer.Sound("assets/collision.wav")
lose_sound = pygame.mixer.Sound("assets/loselife.wav")
font = pygame.font.Font(None, 36)
big_font = pygame.font.Font(None, 80)


class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)

    def update(self):
        self.rect.y += self.speed


class Boss(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = pygame.transform.scale2x(boss_image)
        self.rect = self.image.get_rect()
        self.angle = 0
        self.health = 10

    def update(self):
        self.angle += 0.03
        self.rect.center = (400 + math.cos(self.angle) * 250,
                            150 + math.sin(self.angle) * 60)


def draw_centered(text, font, y):
    image = font.render(text, True, (255, 255, 255))
    screen.blit(image, image.get_rect(center=(400, y)))


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))
score = 0
lives = 3
boss = None
state = "menu"

running = True
while running:
    for event in pygame.event.get():
```

```
        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RETURN and state not in ("playing", "paused"):
                state = "playing"
                player.midbottom = (400, 590)
                score = 0
                lives = 3
                bullets = []
                boss = None
                for enemy in enemies:
                    enemy.reset()
            elif event.key == pygame.K_p and state == "playing":
                state = "paused"
            elif event.key == pygame.K_p and state == "paused":
                state = "playing"

    if state == "playing":
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT]:
            player.x -= speed
        if keys[pygame.K_RIGHT]:
            player.x += speed
        player.clamp_ip(screen.get_rect())

        if cooldown > 0:
            cooldown -= 1
        if keys[pygame.K_SPACE] and cooldown == 0:
            bullets.append(bullet_image.get_rect(midbottom=player.midtop))
            shoot_sound.play()
            cooldown = 15

        for bullet in bullets:
            bullet.y -= 10
        bullets = [b for b in bullets if b.bottom > 0]
        enemies.update()

        for enemy in enemies:
            hit = enemy.rect.collidelist(bullets)
            if hit != -1:
                bullets.pop(hit)
                enemy.reset()
                score += 10
                hit_sound.play()
            elif enemy.rect.top > 600 or enemy.rect.colliderect(player):
                enemy.reset()
                lives -= 1
                lose_sound.play()
        if boss is None and score >= 100:
            boss = Boss()
        if boss is not None:
            boss.update()
            hit = boss.rect.collidelist(bullets)
            if hit != -1:
                bullets.pop(hit)
                boss.health -= 1
                hit_sound.play()
                if boss.health <= 0:
                    score += 500
                    state = "won"
```

```
        if lives <= 0:
            state = "game_over"

    screen.blit(background, (0, 0))
    if state == "menu":
        draw_centered("Most Wanted UFO Mini", big_font, 250)
        draw_centered("Press ENTER to play", font, 340)
    else:
        enemies.draw(screen)
        for bullet in bullets:
            screen.blit(bullet_image, bullet)
        screen.blit(player_image, player)
        hud = font.render(f"Score: {score}   Lives: {lives}", True, (255, 255, 255))
        screen.blit(hud, (10, 10))
    if state == "paused":
        draw_centered("PAUSED", big_font, 300)
    if state == "game_over":
        draw_centered("GAME OVER", big_font, 250)
        draw_centered(f"Score: {score}  -  ENTER to play again", font, 340)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Ganar es un estado más, con su propia pantalla

Falta que el jefe se vea. En el dibujo, después del ovni, dibuja al jefe solo si existe y todavía no ganaste. Así, cuando lo vences, desaparece de la pantalla. Y al final, igual que en la pantalla de fin, si state vale won, escribe YOU WIN con la fuente grande, y debajo el puntaje. Aquí Enter también funciona, porque empieza una partida desde cualquier estado que no sea playing ni paused.

Tu `game.py` queda así — las líneas resaltadas son las nuevas o cambiadas (archivo `paso-4.py`):

```
import math
import random

import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
boss_image = pygame.image.load("assets/boss.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
hit_sound = pygame.mixer.Sound("assets/collision.wav")
lose_sound = pygame.mixer.Sound("assets/loselife.wav")
font = pygame.font.Font(None, 36)
big_font = pygame.font.Font(None, 80)


class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
```

```
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)

    def update(self):
        self.rect.y += self.speed


class Boss(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = pygame.transform.scale2x(boss_image)
        self.rect = self.image.get_rect()
        self.angle = 0
        self.health = 10

    def update(self):
        self.angle += 0.03
        self.rect.center = (400 + math.cos(self.angle) * 250,
                            150 + math.sin(self.angle) * 60)


def draw_centered(text, font, y):
    image = font.render(text, True, (255, 255, 255))
    screen.blit(image, image.get_rect(center=(400, y)))


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))
score = 0
lives = 3
boss = None
state = "menu"

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RETURN and state not in ("playing", "paused"):
                state = "playing"
                player.midbottom = (400, 590)
                score = 0
                lives = 3
                bullets = []
                boss = None
                for enemy in enemies:
                    enemy.reset()
            elif event.key == pygame.K_p and state == "playing":
                state = "paused"
            elif event.key == pygame.K_p and state == "paused":
                state = "playing"
```

```
    if state == "playing":
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT]:
            player.x -= speed
        if keys[pygame.K_RIGHT]:
            player.x += speed
        player.clamp_ip(screen.get_rect())

        if cooldown > 0:
            cooldown -= 1
        if keys[pygame.K_SPACE] and cooldown == 0:
            bullets.append(bullet_image.get_rect(midbottom=player.midtop))
            shoot_sound.play()
            cooldown = 15

        for bullet in bullets:
            bullet.y -= 10
        bullets = [b for b in bullets if b.bottom > 0]
        enemies.update()

        for enemy in enemies:
            hit = enemy.rect.collidelist(bullets)
            if hit != -1:
                bullets.pop(hit)
                enemy.reset()
                score += 10
                hit_sound.play()
            elif enemy.rect.top > 600 or enemy.rect.colliderect(player):
                enemy.reset()
                lives -= 1
                lose_sound.play()
        if boss is None and score >= 100:
            boss = Boss()
        if boss is not None:
            boss.update()
            hit = boss.rect.collidelist(bullets)
            if hit != -1:
                bullets.pop(hit)
                boss.health -= 1
                hit_sound.play()
                if boss.health <= 0:
                    score += 500
                    state = "won"
        if lives <= 0:
            state = "game_over"

    screen.blit(background, (0, 0))
    if state == "menu":
        draw_centered("Most Wanted UFO Mini", big_font, 250)
        draw_centered("Press ENTER to play", font, 340)
    else:
        enemies.draw(screen)
        for bullet in bullets:
            screen.blit(bullet_image, bullet)
        screen.blit(player_image, player)
        if boss is not None and state != "won":
            screen.blit(boss.image, boss.rect)
        hud = font.render(f"Score: {score}   Lives: {lives}", True, (255, 255, 255))
        screen.blit(hud, (10, 10))
```

```
    if state == "paused":
        draw_centered("PAUSED", big_font, 300)
    if state == "game_over":
        draw_centered("GAME OVER", big_font, 250)
        draw_centered(f"Score: {score}  -  ENTER to play again", font, 340)
    if state == "won":
        draw_centered("YOU WIN!", big_font, 250)
        draw_centered(f"Score: {score}  -  ENTER to play again", font, 340)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Una partida completa contra el jefe

Ejecuta el juego, y juega una partida completa. Al llegar a cien puntos aparece el jefe, dando vueltas arriba. Cada acierto le quita vida; los enemigos siguen cayendo, así que no te descuides. Y cuando su vida llega a cero: ganaste.

Pruébalo:

```
python game.py
```

El juego original usa las mismas ideas

Most Wanted UFO Mini se basa en Most Wanted UFO Classic, un juego de código abierto. Este es su archivo principal: mira cómo hace el original al jefe. Mira estas líneas: es el jefe, moviéndose en círculo. Ya reconoces las piezas: un ángulo que crece en cada cuadro, el centro de la pantalla, un radio de cien, el coseno para la equis y el seno para la y. Lo único nuevo es math.radians: aquí el ángulo va en grados, y esa función lo convierte a radianes, la misma unidad que usaste en tu jefe. Una vuelta completa son trescientos sesenta grados, o unos seis coma dos ocho radianes. El original está escrito de otra forma, con todo dentro de una función grande, pero las ideas son las mismas que aprendiste.

El código del juego original está en `original/main.py`, junto a este curso.

Terminaste Most Wanted UFO Mini

Terminaste el curso, y construiste un juego completo, desde una ventana vacía. Usaste el bucle de juego, el teclado, listas, y clases. Agregaste choques, puntos, vidas, y estados. Y moviste un jefe con seno y coseno. Tu proyecto final es un mod: descarga Most Wanted UFO Classic, y cambia algo del jefe, como su radio, su velocidad, o su vida. Los gráficos y sonidos son libres, siempre que des crédito a su autor. Gracias por jugar, y por programar.

- Bucle, teclado, listas y clases
- Choques, puntos, vidas y estados
- Seno y coseno para moverse en círculo
- Tu mod: cambia el juego original

Resumen

- `math.cos(angulo)` y `math.sin(angulo)` van de -1 a 1; multiplicados por un radio y sumados a un centro, recorren un círculo (o un óvalo, con radios distintos).
- Si el ángulo crece un poco en cada cuadro, el personaje da vueltas; `0.03` por cuadro es una vuelta cada 3.5 segundos a 60 FPS.
- `boss = None` significa "todavía no hay jefe"; `if boss is not None:` evita usarlo antes de que exista.
- El jefe tiene su propia vida (`self.health`); al llegar a 0 el estado pasa a `"won"`.
- Most Wanted UFO Classic usa las mismas ideas en `main.py`, líneas 166 a 170: `math.cos` y `math.sin` con un radio de 100.
- **Para tu mod:** el código del juego original es MIT y sus gráficos y sonidos son CC BY 4.0: puedes modificarlos y publicarlos dando crédito a Jose Daniel Soto.

Ejercicio

Proyecto final: tu mod.

- Descarga Most Wanted UFO Classic: `git clone https://github.com/Magic-Lab-Studio/most-wanted-ufo-classic`.
- Crea un entorno, instala lo que pide `requirements.txt` y ejecuta `python main.py`.
- Abre `main.py` y busca el jefe (líneas 166 a 170). Cambia al menos una cosa: el radio (`100`), la velocidad del giro (`self.boss_speed` en `game_manager.py`) o la vida del jefe (`self.boss_health`).
- En tu Most Wanted UFO Mini, haz que el jefe dispare: cada 60 cuadros agrega una bala que baje desde `boss.rect.midbottom`, y si toca al ovni, resta una vida. Pista: el original ya lo hace en las líneas 172 a 175 de `main.py`.
- Si publicas tu mod, da crédito: "Basado en Most Wanted UFO Classic, de Jose Daniel Soto (CC BY 4.0)".

Solución

Para el punto 4, en Most Wanted UFO Mini:

```
boss_bullets = []
boss_timer = 0
# dentro de if boss is not None:
boss_timer += 1
if boss_timer >= 60:
    boss_bullets.append(bullet_image.get_rect(midtop=boss.rect.midbottom))
    boss_timer = 0
for b in boss_bullets:
    b.y += 6
hit = player.collidelist(boss_bullets)
if hit != -1:
    boss_bullets.pop(hit)
    lives -= 1
    lose_sound.play()
boss_bullets = [b for b in boss_bullets if b.top < 600]
```

Pon `boss_bullets = []` y `boss_timer = 0` antes del bucle, junto a `boss = None`, y repítelos en el bloque de Enter que empieza la partida. Dibuja cada bala de `boss_bullets` junto con las demás.

Basado en Most Wanted UFO Classic, de Jose Daniel Soto: código MIT, gráficos y sonidos CC BY 4.0.
Un curso de Magic Lab Studio · magiclabstudio.com